# 0. First 30-45 Minutes (Planning)

## Documentation

- Problem Understanding
- Objectives
- User Roles
- Functional Requirements
- Non-functional Requirements
- Assumptions
- Constraints
- Tech Stack
- Module List
- Database Design
- Architecture Diagram
- API Contract
- Folder Structure
- Git Plan
- Timeline

Only after this → coding.

---

# 1. Feature Prioritization

## Must Have ⭐⭐⭐⭐⭐

- Login/Auth
- Dashboard
- Departments
- Employees
- Asset Categories
- Asset Registration
- Asset Allocation
- Resource Booking
- Maintenance Workflow
- Audit Workflow
- Reports

- Notifications

---

## Nice To Have ⭐⭐⭐

- QR Scanner
- Asset Images
- Email Notifications
- Advanced Analytics

---

# 2. Modular Architecture

Presentation Layer
│
├── Authentication Module
├── Dashboard Module
├── Organization Module
├── Asset Module
├── Allocation Module
├── Booking Module
├── Maintenance Module
├── Audit Module
├── Reports Module
├── Notification Module
│
Service Layer
│
├── Auth Service
├── Asset Service
├── Booking Service
├── Maintenance Service
├── Audit Service
│
Repository Layer
│
├── PostgreSQL

Every module is independent.

---

# 3. Database Design (Biggest Focus)

Entities

Users
Roles
Departments
Employees
AssetCategory
Assets
Allocations
Transfers
Bookings
MaintenanceRequests
AuditCycles
AuditRecords
Notifications
ActivityLogs

Relationships

Department
    |
    ├── Employees
    |
    └── Assets

Assets
    |
    ├── Maintenance
    ├── Allocation
    ├── Booking
    └── Audit

We'll normalize to **3NF**.

---

# 4. ER Diagram

Department
    |
Employees ---- Roles

```
    |
Allocations ---- Assets ---- Categories
    |            |
Transfers      Maintenance
    |
Bookings
    |
AuditCycle
    |
AuditRecord
```

Very detailed.

Judges LOVE this.

---

# 5. API Contract

Before coding.

Example

POST /login

POST /signup

GET /dashboard

GET /assets

POST /assets

PUT /assets/:id

DELETE /assets/:id

POST /allocation

POST /booking

POST /maintenance

POST /audit

Frontend knows these.

Backend builds these.

---

//ADDED ALL DB DESIGN AND API CONTRACT

# DATABASE DESIGN

---

## 1. roles

role_id (PK)
role_name

Values

Admin
Asset Manager
Department Head
Employee
Auditor

---

## 2. departments

department_id (PK)
department_name
parent_department_id (FK -> departments)
department_head_id (FK -> employees)
status
created_at

---

## 3. employees

employee_id (PK)
name
email (Unique)
password
department_id (FK)
role_id (FK)
status

created_at

---

## 4. asset_categories

category_id (PK)
category_name
description
default_warranty_months
created_at

---

## 5. assets

asset_id (PK)
asset_tag (Unique)
asset_name
category_id (FK)
serial_number
purchase_date
purchase_cost
condition
location
status
is_shared
image
created_at

Status Enum

Available
Allocated
Reserved
Under Maintenance
Lost
Retired
Disposed

---

## 6. asset_allocations

allocation_id (PK)
asset_id (FK)
employee_id (FK)

department_id (FK)
allocated_date
expected_return
returned_date
condition_return
status

Status

Allocated
Returned
Overdue
Transferred

---

# 7. transfer_requests

transfer_id (PK)
allocation_id (FK)
requested_by
from_employee
to_employee
approved_by
status
requested_date
approved_date
remarks

Status

Pending
Approved
Rejected

---

# 8. resource_bookings

booking_id (PK)
asset_id (FK)
employee_id (FK)
purpose
start_datetime
end_datetime
status
created_at

Status

Upcoming
Ongoing
Completed
Cancelled

---

## 9. maintenance_requests

maintenance_id (PK)
asset_id (FK)
raised_by
description
priority
photo
approved_by
technician
status
created_at
completed_at

Status

Pending
Approved
Rejected
Assigned
In Progress
Resolved

---

## 10. audit_cycles

audit_cycle_id (PK)
title
department_id
location
start_date
end_date
status
created_by

Status

Upcoming
Running
Closed

---

## 11. audit_records

audit_record_id (PK)
audit_cycle_id (FK)
asset_id (FK)
auditor_id
verification_status
remarks
verified_date

Verification Status

Verified
Missing
Damaged

---

## 12. notifications

notification_id (PK)
employee_id (FK)
title
message
type
is_read
created_at

---

## 13. activity_logs

log_id (PK)
employee_id (FK)
action
module
reference_id
timestamp

---

# Relationships

Role
|
Employees
|
Departments
|
Assets
|
Asset Allocation
|
Transfer Request

Assets
|
Bookings

Assets
|
Maintenance

Assets
|
Audit Records
|
Audit Cycle

Employees
|
Notifications

Employees
|
Activity Logs

---

# API CONTRACT

## Authentication

POST /api/auth/login

POST /api/auth/logout

GET /api/auth/profile

---

# Dashboard

GET /api/dashboard

Returns

Assets Available

Assets Allocated

Maintenance Today

Upcoming Returns

Bookings Today

Pending Requests

---

# Departments

GET /api/departments

POST /api/departments

PUT /api/departments/:id

DELETE /api/departments/:id

---

# Employees

GET /api/employees

POST /api/employees

PUT /api/employees/:id

DELETE /api/employees/:id

---

## Asset Categories

GET /api/categories

POST /api/categories

PUT /api/categories/:id

DELETE /api/categories/:id

---

## Assets

GET /api/assets

GET /api/assets/:id

POST /api/assets

PUT /api/assets/:id

DELETE /api/assets/:id

---

## Asset Allocation

GET /api/allocations

POST /api/allocate

POST /api/return

POST /api/transfer

GET /api/allocations/history

---

## Resource Booking

GET /api/bookings

POST /api/bookings

PUT /api/bookings/:id

DELETE /api/bookings/:id

GET /api/bookings/calendar

---

## Maintenance

GET /api/maintenance

POST /api/maintenance

PUT /api/maintenance/:id

GET /api/maintenance/history

---

## Audit

GET /api/audits

POST /api/audits

POST /api/audits/verify

PUT /api/audits/:id/close

---

## Reports

GET /api/reports/assets

GET /api/reports/departments

GET /api/reports/maintenance

GET /api/reports/audits

GET /api/reports/bookings

---

## Notifications

GET /api/notifications

PUT /api/notifications/read/:id

---

## Activity Logs

GET /api/logs

# ✅ Final Notes

- **13 normalized tables** (3NF)
- No duplicate data
- Every table has proper foreign keys
- Supports all required workflows:
    - Asset lifecycle
    - Conflict-free allocation
    - Booking overlap validation
    - Maintenance approval workflow
    - Audit cycles
    - Notifications
    - Activity logs
- Matches every required screen and workflow in the AssetFlow problem statement.

# 6. Folder Structure

project/

frontend/

components/

pages/

services/

hooks/

backend/

controllers/

routes/

models/

middleware/

services/

database/

schema.sql

seed.sql

docs/

README.md

ERD.png

Architecture.png

---

# 7. Task Split

## Hrishita

Frontend

Dashboard

Assets UI

Booking UI

Reports UI

Authentication UI

API Integration

---

## Hannah

Database

Backend

Authentication

Booking Logic

Maintenance Logic

Audit Logic

Notifications

---

Shared

Architecture

ER Diagram

Testing

Presentation

---

# 8. Git Workflow

Only **main**.

Every hour

git pull origin main

work

git add .

git commit -m "Meaningful message"

git push origin main

Never edit same file.

---

# 9. Backend Architecture

Controller

↓

Service

↓

Repository

↓

Database

NO business logic inside routes.

Everything modular.

---

# 10. Database Design Philosophy

No duplicate data

Foreign Keys

Indexes

Constraints

Unique Keys

Transactions

Proper normalization

This is where many teams lose.

We'll make ours beautiful.

---

# 11. UI Philosophy

Minimal

Responsive

Professional

No clutter

ERP dashboard look

Cards

Tables

Charts

Side Navigation

Search everywhere

Filters everywhere

---

# 12. Security

Role Based Access

JWT

Password Hashing

Input Validation

Parameterized Queries

Session Management

Authorization Middleware

---

# 13. Scalability

Modules independent

Reusable components

Reusable APIs

Separate services

Repository Pattern

Future microservices possible

---

# 14. Prototype

❌ No mock data.

Everything from PostgreSQL.

Every screen functional.

Every workflow connected.

---

# 15. Final Deliverable

- ✅ Documentation
- ✅ Architecture Diagram

- ✅ ER Diagram
- ✅ PostgreSQL Database
- ✅ REST APIs
- ✅ Responsive UI
- ✅ Modular Code
- ✅ Git History (both committing hourly)
- ✅ Real Working Prototype
- ✅ Clean README

Perfect. **This is exactly the level of documentation I'd make before coding.** It takes about **30-40 minutes**, and then you and Hannah can code independently.

---

LMAOO 😭😭 My bad baka. I switched back to the ESG problem.

For the **AssetFlow (Asset & Resource Management System)**, **this** is the documentation we'll actually make before coding.

---

# AssetFlow - Enterprise Asset & Resource Management System

# 1. Problem Statement

Organizations struggle to efficiently manage assets, resource bookings, maintenance schedules, and audits due to fragmented systems, manual tracking, duplicate records, and poor visibility.

The objective is to build a centralized Asset Management Platform that enables organizations to efficiently manage their assets, employees, bookings, maintenance, audits, and reports through a scalable ERP solution.

---

# 2. Objectives

### Business Objectives

- Centralize Asset Management
- Prevent Asset Mismanagement
- Track Asset Lifecycle
- Simplify Resource Booking
- Improve Maintenance Management
- Generate Reports

### Technical Objectives

- Modular Architecture
- PostgreSQL Database
- REST APIs
- Responsive UI
- Role Based Access
- Secure Authentication
- No Mock Data

---

# 3. User Roles

## Administrator

- Manage Departments
- Manage Employees
- Manage Assets
- Generate Reports
- Configure System

---

## Employee

- Request Assets
- Book Resources
- Return Assets
- Raise Maintenance Requests

---

## Manager

- Approve Requests
- View Department Assets
- Track Usage

---

## Auditor

- Conduct Audits
- Verify Assets
- Generate Audit Reports

---

# 4. Functional Requirements

## Authentication

- Login
- Logout
- RBAC

---

## Dashboard

- Total Assets
- Available Assets
- Allocated Assets
- Under Maintenance
- Upcoming Audits
- Recent Activities

---

## Department Module

- CRUD Departments

---

## Employee Module

- CRUD Employees

---

## Asset Category Module

- CRUD Categories

---

## Asset Module

- Register Asset
- Edit Asset
- Delete Asset
- Asset History
- Asset Status

---

## Asset Allocation Module

- Allocate Asset
- Return Asset
- Transfer Asset

---

## Booking Module

- Resource Booking
- Conflict Detection
- Booking Approval

---

## Maintenance Module

- Maintenance Requests
- Approval Workflow
- Completion Status

---

## Audit Module

- Audit Scheduling
- Audit Reports
- Compliance Tracking

---

## Notification Module

- Allocation Alerts
- Booking Approval
- Maintenance Reminder
- Audit Reminder

---

## Reports

- Asset Report
- Department Report
- Maintenance Report
- Audit Report

---

# 5. Non Functional Requirements

- Responsive UI
- Secure Authentication
- PostgreSQL
- Modular Code
- High Performance
- Scalable Architecture
- Data Integrity

- Responsive Dashboard

---

# 6. Assumptions

- One employee belongs to one department.
- One asset belongs to one category.
- One asset can have multiple maintenance records.
- One asset can have multiple audit records.
- Bookings cannot overlap.
- Deleted assets are archived instead of permanently removed.

---

# 7. Constraints

- PostgreSQL Database
- Responsive UI
- Modular Architecture
- Latest Code on main
- Every member commits hourly
- Real Database Integration

---

# 8. Tech Stack

Frontend

- React
- Tailwind

Backend

- Express

Database

- PostgreSQL

Authentication

- JWT

Version Control

- GitHub

---

# 9. High Level Architecture

React

↓

REST API

↓

Express

↓

Services

↓

Repositories

↓

PostgreSQL

---

# 10. Modules

Authentication

Dashboard

Departments

Employees

Asset Categories

Assets

Allocations

Bookings

Maintenance

Audits

Notifications

Reports

Settings

---

# 11. Main Database Tables

Users

Roles

Departments

Employees

AssetCategories

Assets

AssetAllocations

AssetTransfers

Bookings

MaintenanceRequests

Audits

AuditResults

Notifications

ActivityLogs

---

# 12. Folder Structure

frontend/

backend/

database/

docs/

---

# 13. API List

Authentication

POST /login

Departments

GET /departments
POST /departments

Employees

GET /employees
POST /employees

Assets

GET /assets
POST /assets
PUT /assets/:id
DELETE /assets/:id

Allocation

POST /allocate
POST /return
POST /transfer

Bookings

POST /booking
PUT /booking/:id

Maintenance

POST /maintenance
PUT /maintenance/:id

Audit

POST /audit
GET /audit

Reports

GET /reports/assets
GET /reports/maintenance
GET /reports/audit

---

# 14. Security

- JWT
- BCrypt

- RBAC
- Input Validation
- SQL Injection Prevention

---

# 15. Task Split

### Hrishita

- Dashboard
- Authentication UI
- Asset UI
- Reports UI
- API Integration

---

### Hannah

- PostgreSQL
- Backend
- Asset Logic
- Booking Logic
- Maintenance Logic
- Audit Logic

---

### Together

- Database
- Architecture
- Testing
- Final Integration

---

# 16. Git Workflow

git pull origin main

↓

Code

↓

git add .

↓

git commit -m "Meaningful message"

↓

git push origin main

---

# 17. Coding Order

1. Folder Structure
2. Database Schema
3. Backend APIs
4. Frontend Pages
5. Integration
6. Remove Mock Data
7. Testing
8. Final Demo

---

## 💯 One thing I would add (and I think this will make your submission stand out)

Before even opening VS Code, we'll spend **10–15 minutes** making a **Module Responsibility Matrix**:

| Module | Owner | Files |
| --- | --- | --- |
| Authentication | Hrishita | `frontend/auth/*` |

| | | |
|---|---|---|
| Dashboard | Hrishita | `frontend/dashboard/*` |
| Assets | Hannah | `backend/assets/*` |
| Database | Hannah | `database/*` |
| Reports | Hrishita | `frontend/reports/*` |
| Notifications | Hannah | `backend/notifications/*` |

This tiny table will **almost eliminate merge conflicts** and let both of you commit every hour while keeping `main` up to date. It's one of the highest ROI things you can do in a hackathon.